

Spring Boot — 复习要点

CH05 | 软件开发架构平台 | 计算机学院

1 内容回顾：Spring MVC 请求处理流程 (Spring MVC Request Flow)

- 请求首先到达前端控制器 (Front Controller / DispatcherServlet)，委托给具体的控制器处理请求。[p.3]
- 前端控制器通过查询处理器映射 (Handler Mapping)，找到 URL 对应的控制器。[p.3]
- 控制器处理请求，包括处理数据、调用业务逻辑等。[p.3]
- 控制器将模型数据（打包）和逻辑视图名返回给前端控制器。[p.3]
- 视图解析器 (View Resolver) 将逻辑视图名匹配成具体的视图实现。[p.3]
- 视图进行模型数据和视图实现的渲染 (Rendering)。[p.3]
- 交付模型数据，给出 Web 响应。[p.3]

2 Spring Boot 概述 (Overview of Spring Boot)

2.1 为什么需要 Spring Boot? (Why Spring Boot?)

- Spring 框架相关组件使用的复杂性：[p.5]
 - 几乎所有 Spring 组件或技术都基于 Spring IoC 和 Spring AOP。[p.5]
 - 每个组件或技术又有自身的相关配置。[p.5]
 - Web 容器和数据库等还有一些其他相关配置。[p.5]
 - 导致一般在使用 Spring 框架相关技术时，“搭环境”往往比“写代码”更耗时、更容易出错。[p.5]

2.2 什么是 Spring Boot? (What is Spring Boot?)

- Spring Boot 是 Spring 为简化 Spring 框架的使用，推出的一个组件/工具。[p.6]
- Spring Boot 是一个基于 Spring 的快速开发脚手架 (Scaffold)，其核心设计哲学是“约定优于配置” (Convention over Configuration)。[p.6]
- Spring Boot 不是对 Spring 框架功能上的替代，而是对 Spring 使用方式的简化。[p.6]
- Spring Boot 本身也不是类似于 Spring MVC 具有某种功能的 Spring 组件。[p.6]
- 官网：<https://spring.io/projects/spring-boot/> [p.7]
- 目前最新稳定版为 4.0.3 (Spring 7.x、JDK 17+)。[p.7]

2.3 Spring Boot 的核心目标 (Core Goals)

- 极低的学习成本和极大的提高开发效率。[p.7]
- 开发可独立运行的 Web 应用。[p.7]
- 简单的组件依赖，自动发现与自动装配 (Auto-Discovery & Auto-Wiring)。[p.7]
- 提供运行时的应用监控 (Runtime Monitoring)。[p.7]
- 提供与分布式架构、云原生架构和大数据等组件的良好集成。[p.7]

2.4 传统 Spring Web 应用开发流程 vs Spring Boot 脚手架

- 传统 Spring Web 应用开发流程：配置环境 → 创建工程 → 构建目录结构 → 设置组件参数 → 配置 Web 容器 → 组件依赖管理 → 业务开发 → 测试与构建 → 部署与发布 → 运维与监控。

[p.8]

- **Spring Boot 脚手架的解决思路:** [p.9]
 - 创建工程与构建目录结构: 使用 Spring Initializr 简化操作。[p.9]
 - 设置组件参数、配置 Web 容器、组件依赖管理: 由 Spring Boot 自动完成。[p.9]
 - 业务开发、测试与构建、部署与发布: 开发人员专注业务。[p.9]
 - 运维与监控: 通过 Spring Actuator 工具提供。[p.9]

3 Spring Boot 快速入门 (Quick Start)

3.1 利用 IntelliJ IDEA 快速构建 Spring Boot 应用

- 使用 IntelliJ IDEA 内置的 Spring Initializr 向导快速构建 Spring Boot 应用。[p.10]
- 选择项目所需的组件 (如 Web、JPA、Security 等 Starter), 自动生成项目骨架。[p.11]

3.2 项目目录结构 (Project Directory Structure)

- Spring Boot 约定的 Web 项目目录结构: 直接运行的 main 方法、集成所有配置的默认 application.properties [p.12]
- 标准目录布局: [p.13]
 - /src/main/java — Java 源代码目录。[p.13]
 - /src/main/resources — 资源目录。[p.13]
 - /src/main/resources/static — 静态资源目录。[p.13]
 - /src/main/resources/templates — 表示层页面目录。[p.13]
 - /src/main/resources/application.properties — Spring Boot 配置文件。[p.13]
 - /src/test — 测试文件目录。[p.13]

3.3 运行项目 (Run the Project)

- 直接运行 main 方法, 无需部署到外部服务器。[p.14]

4 Spring Boot 自动配置原理 (Auto-Configuration Principles)

4.1 四大组成 (Four Pillars)

Spring Boot 的自动化和“开箱即用”(Out-of-the-Box) 主要由以下四方面组成: [p.16]

- 依赖管理的 Starter 机制。[p.16]
- Spring IoC 自动配置 Bean 机制。[p.16]
- 统一集成的配置文件。[p.16]
- 内嵌式 Servlet 容器。[p.16]

4.2 Starter 机制 (Starter Mechanism)

- 基于 Maven 提供简化和统一的依赖管理。[p.17]
- `spring-boot-starter-parent`: 每个项目都可以继承的父 POM。[p.17]
- 该父配置中定义了各种常用依赖的版本和关系, 确保了项目中各种第三方依赖的兼容性和依赖关系。[p.17]

4.2.1 官方维护的常用 Starter [p.19]

- `spring-boot-starter-web` — 包含 Spring MVC、Jackson、Validation、内嵌 Tomcat。[p.19]
- `spring-boot-starter-data-jpa` — 集成 Spring Data JPA、Hibernate、数据库驱动; 提供

JpaRepository。[p.19]

- `spring-boot-starter-security` — 集成 Spring Security, 提供默认登录界面与安全配置。[p.19]
- `spring-boot-starter-test` — 集成 JUnit、Mockito、AssertJ、Spring Test 等测试库。[p.19]
- `spring-boot-starter-thymeleaf` — 集成 Thymeleaf 模板引擎与解析器。[p.19]
- `spring-boot-starter-aop` — 启用 Spring AOP 自动代理。[p.19]
- `spring-boot-starter-actuator` — 提供监控端点 (Endpoints) 和指标收集 (Metrics)。[p.19]

4.3 自动配置机制 (Auto-Configuration Mechanism)

- 自动配置是 Spring Boot 自动化中的核心机制。[p.20]
- 基本原理: 大量的条件判断 + 大量的默认值。[p.20]
- 工作流程: [p.20]
 1. 启动: 从 `@SpringBootApplication` 到 `@EnableAutoConfiguration`。[p.20]
 2. 加载配置: 扫描所有 jar 包下的 `META-INF/spring/XXX.AutoConfiguration.imports` 文件。[p.20]
 3. 条件评估: 读取该文件中列出的所有自动配置类 (如 `DispatcherServletAutoConfiguration`)。[p.20]
 4. 条件注解: 使用 `@ConditionalOnClass` 等条件注解进行判断 (如: 如果 classpath 下有 `DispatcherServlet` 且用户没有自定义 `DispatcherServlet Bean`, 则自动配置一个 `DispatcherServlet`)。[p.20]

4.3.1 源码剖析: 以 `DispatcherServletAutoConfiguration` 为例 [p.21]

1. `@AutoConfiguration`: 标记这是一个自动配置类。[p.21]
2. `@ConditionalOnClass(DispatcherServlet.class)`: 只有存在 `DispatcherServlet` 类才生效 (通常由引用 Web Starter 决定)。[p.21]
3. `@ConditionalOnWebApplication`: 确认这是一个 Web 应用。[p.21]
4. 内部定义 `@Bean` 方法创建 `DispatcherServlet` 和 `DispatcherServletRegistrationBean` (注册 Servlet)。[p.21]
5. `@ConditionalOnMissingBean(DispatcherServlet.class)`: 如果用户自己定义了一个 `DispatcherServlet`, Spring Boot 的自动配置就会失效 (Back off), 以用户的为准。[p.21]

4.4 配置文件 (Configuration Files)

- 配置文件类型: `application.properties` 或 `application.yml` (推荐 YAML)。[p.22]
- 也可以使用 Java Config 的方式: 封装成配置类, 进行单个属性的设置。[p.23]

4.4.1 多环境配置 (Multi-Environment / Profile) [p.24]

- 需求: 开发环境 (dev)、测试环境 (test)、生产环境 (prod) 配置不同 (如端口、日志级别)。[p.24]
- 命名规范: `application-{profile}.yml`, 如 `application-dev.yml`、`application-prod.yml`。[p.24]
- 激活方式: [p.24]
 - 在 `application.yml` 中指定 `spring.profiles.active: dev`。[p.24]
 - 命令行参数激活: `java -jar myapp.jar --spring.profiles.active=prod`。[p.24]

4.5 内嵌式 Servlet 容器 (Embedded Servlet Container)

- Spring Boot 默认使用嵌入式 Tomcat 作为 Servlet 容器, 还支持 Jetty、Undertow 等。[p.25]
- Spring Boot 的 Web 应用无需部署到外部服务器, 直接 `java -jar` 即可运行。[p.25]

- **传统模式 vs Spring Boot 模式:** [p.25]
 - 传统模式: 应用 → 丢进 Tomcat → Tomcat 启动。[p.25]
 - Spring Boot 模式: `main` 方法 → `SpringApplication` → 创建应用上下文 (`ApplicationContext`) → 启动 Tomcat / Jetty → 注册 `DispatcherServlet`。[p.25]
- Spring Boot 通过 Java 代码在运行时动态创建 Tomcat 实例, 并将当前应用注册进去, 因此可以直接通过 `java -jar` 运行。[p.25]

4.5.1 Servlet 容器常用配置 (Servlet Container Configuration) [p.26]

- 修改端口: 默认 HTTP 端口为 8080, 可在配置文件中通过 `server.port` 设置, 也可在环境变量中设置 `SERVER_PORT`。[p.26]
- 关闭 HTTP 服务: 将 `server.port` 设为 -1 可以创建 `WebApplicationContext` 但不打开端口。[p.26]
- 随机端口: 设置 `server.port=0` 让系统自动选取一个可用端口。[p.26]
- 压缩响应: 配置 `server.compression.enabled=true` 启用响应压缩。[p.26]
- 启用 SSL: 通过 `server.ssl.*` 属性提供证书路径和密码。[p.26]

4.6 自动配置小结 (Auto-Configuration Summary) [p.27]

Spring Boot 启动流程概览:

1. 加载配置文件 (`application.properties`)。[p.27]
2. 自动装配 Starter 组件 (`spring-boot-starter-web` 增加 Web 支持, `spring-boot-starter-data` 增加数据库支持, `spring-boot-starter-logging` 增加 Logback 日志支持等)。[p.27]
3. 加载组件 (`@Repository`, `@Controller`, `@Entity...`)。[p.27]
4. 应用初始化。[p.27]

5 项目中使用范例 (Usage Examples in Projects)

5.1 使用 Spring Boot 进行日志管理 (Logging Management)

5.1.1 日志管理简介 (Introduction to Logging)

- 在程序运行过程中, 为监测某些功能或验证某些指标是否正确, 需要输出相关信息, `System.out.println()` 就是最简单的日志处理。[p.29]
- 如 JUL (`java.util.logging`) 日志的使用示例。[p.29]

5.1.2 Java 平台中日志门面和日志实现的概念 [p.30]

- **日志门面 (Logging Facade):** 只提供日志相关的接口定义 (API), 而不提供具体的接口实现。日志门面在使用时可以动态或者静态地指定具体的日志框架实现, 解除了接口和实现的耦合, 使用户可以灵活地选择日志的具体实现框架。常见的日志门面有 Commons-Logging (JCL)、SLF4J。[p.30]
- **日志实现/系统 (Logging Implementation):** 与日志门面对应, 提供了具体的日志接口实现, 应用程序通过它执行日志打印的功能。常见的日志实现有 Log4j、JUL、Logback、Log4j2。[p.30]

5.1.3 Spring Boot 中的日志管理 [p.32]

- Spring Boot 默认使用 **SLF4J + Logback** 作为日志解决方案。[p.32]
- 通过配置文件 (`application.yml` 或 `application.properties`) 对日志进行自定义的设置 (如日志级别、输出格式、文件路径等)。[p.32]

5.2 使用 Spring Boot 进行事务管理 (Transaction Management)

5.2.1 事务管理简介：事务的 ACID 属性 [p.33]

- **场景：**A 账户向 B 账户转账 100 元。步骤 1：A 账户扣款 100 元；步骤 2：B 账户加款 100 元。问题：步骤 1 成功，步骤 2 失败（网络中断、服务器宕机），钱就“消失”了。解决方案：要么全部成功，要么全部失败 — 这就是事务。[p.33]
- **原子性 (Atomicity)：**事务中的所有操作作为一个整体，不可分割。[p.33]
- **一致性 (Consistency)：**事务执行前后，数据完整性约束没有被破坏。[p.33]
- **隔离性 (Isolation)：**多个事务并发执行时，彼此互不干扰。[p.33]
- **持久性 (Durability)：**事务一旦提交，对数据的修改是永久性的。[p.33]

5.2.2 编程式事务 (Programmatic Transaction) [p.34]

- Spring 中提供 `TransactionTemplate` 或 `PlatformTransactionManager` 手动控制事务。[p.34]
- 编程式事务灵活，可细粒度控制，但是代码冗余，与业务逻辑耦合。[p.34]

5.2.3 声明式事务 (Declarative Transaction) [p.35]

- Spring 中基于 AOP，通过 `@Transactional` 注解实现声明式事务。[p.35]
- 声明式事务业务代码干净，无侵入，实际项目开发中大部分场景使用声明式事务。[p.35]

5.2.4 何时使用编程式事务？(When to Use Programmatic Transaction?) [p.36]

- 当需要细粒度事务控制的特殊场景（如部分提交、嵌套事务等）时考虑使用编程式事务。[p.36]

本章小结 (Chapter Summary) [p.37]

- **Spring Boot 概述：**为什么需要 Spring Boot？Spring Boot 是什么？[p.37]
- **Spring Boot 自动配置原理：**Starter 机制、自动配置、配置文件的使用、内嵌 Tomcat 容器的使用。[p.37]
- **项目中使用范例：**实际项目中通过 Spring Boot 进行日志管理（SLF4J + Logback）和事务管理（编程式与声明式事务）。[p.37]